

# CS2422 Assembly Language & System Programming

September 23, 2003

## Today's Topics

- Data Declaration
- Basic Instructions

## Study Guide

- Section 3.4: Data Definition.
- Section 3.5: Constants.
- Sections 4.1: Data Transfer Instructions.

## Example of Last Week

```
TITLE Add and Sub
; This is a comment
INCLUDE Irvine32.inc

.code
Main proc
    MOV EAX, 10000h
    ADD EAX, 40000h
    SUB EAX, 20000h
    CALL DumpRegs
    EXIT
Main endp
End Main
```

## Another Example

```
INCLUDE Irvine32.inc
C1 EQU 40000h
.data
Var1 DWORD 10000h

.code
Main proc
    MOV EAX, Var1
    ADD EAX, C1
    CALL DumpRegs
    EXIT
Main endp
End Main
```

## Data Declaration

- [Label], Type, Initialization (or just ?)
  - Example: Var1 BYTE 7
- Use ? if no initialization necessary.
  - Example: Var1 BYTE ?
- Other data types: (See Table 3-2 in p.80)
  - WORD (or DW),
  - DWORD (or DD), ...etc.

## Signed vs. Unsigned

- Signed vs. Unsigned:
  - SBYTE vs. BYTE
  - SWORD vs. WORD
  - ...etc.
- Example:
  - Var1 BYTE 255
  - Var1 SBYTE -1

## Characters and Strings

- How about characters?
- A few examples:
  - Var1 BYTE 'A'
  - Var1 BYTE 41h
  - S1 BYTE 'Hello, World!'
  - Line BYTE '-----'

## How About A List or Array?

- Just list them! For example:
  - List1 BYTE 10, 32, 41h
- You may also mix them:
  - List1 BYTE 10, 32, ?, 'A'
  - List2 BYTE "good", 'o', 'r', 'b', 'a', 'd', 0
- So, Is List2 an Array?
  - Yes and No! (e.g., List2 actually means 'g'.)

## DUP

- Good for allocating space for a string or array.
- Examples:
  - Var1 BYTE 20 DUP (0)
  - Var2 BYTE 20 DUP (?)
  - Var3 BYTE 4 DUP ("STACK")

## EQU is for Constants

- EQU for constants.
- Also OK to use = for integers.
- Examples:
  - COUNT = 500
  - COUNT EQU 500

## Expression and Text in EQU

- OK to use expressions in EQU:
  - Matrix1 EQU 10 \* 10
  - Matrix1 EQU <10 \* 10>
- No expression evaluation if within < >.
- EQU accepts texts too:
  - Msg EQU <“Press any key...”, 0>

- Why EQU? Why not storing the constants as variables (in memory)?
- Exercise: Does it save memory? Isn't the constant stored somewhere in memory anyway? (*Hint: See Week02a slides.*)

*A few fine points  
(that are skipped previously on purpose)...*

## If Irvine32.inc Is Not Used...

```
TITLE Add and Subtract
.386
.MODEL flat,stdcall
.STACK 4096
ExitProcess PROTO,dwExitCode:DWORD
DumpRegs PROTO

.code
main PROC
...
```

```
.code
main PROC
    mov eax,10000h
    add eax,40000h
    sub eax,20000h
    call DumpRegs
    INVOKE ExitProcess,0
main ENDP
END main
```

*Entering Chapter 4...*

## Data Transfer Instructions

- MOV is for moving data between:
  - Memory
  - Register
  - Immediate (constant)
- Almost all combinations, except:
  - Memory to Memory!

## Memory to Memory?

- Must go through a register...

```
.data
Var1 WORD?
Var2 WORD ?
.code
    MOV AX, var1
    MOV var2, AX
```

## Zero or Sign Extension

- What happens to ECX if -1 is moved to CX?
  - Are the higher 16 bits of ECX all 0?
  - What number does ECX represent now?
- The solution: MOVZX and MOVSX
  - MOVZX always fills higher bits with 0.
  - MOVSX fills higher bits by “sign extension”.
  - Just extend the left-most bit!

## LAHF/SAHF and XCHG

- LAHF to load flags into AH.
- SAHF to go the other way.
- XCHG for exchange data between:
  - Register, register
  - Register, memory
  - Memory, register(again, no memory to memory)

## Direct-Offset Operands

- Adding a displacement (or offset) to a variable name:

```
arrayB BYTE 10h, 20h, 30, 40h, 50h
...
MOV AL, arrayB      ; AL=10h
MOV AL, [arrayB+1]  ; AL=20h
MOV AL, arrayB+1    ; Is it valid?
```